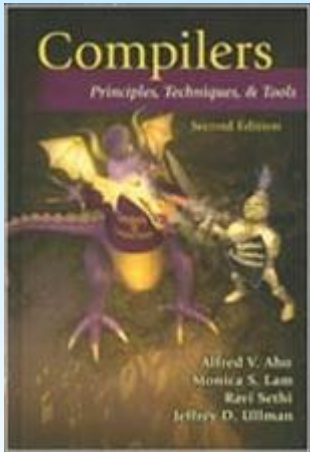
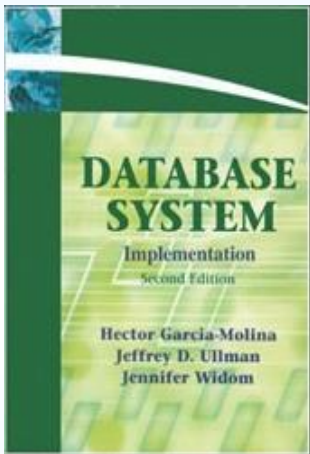




编译原理上机辅导

DBMS的设计与实现



西安电子科技大学 计算理论与技术研究所

王小兵

xbwang@foxmail.com



国产数据库

👉 华为openGauss





国产数据库

👉 阿里PolarDB





目录

1. 上机题目
2. 改进建议
3. 验收方式
4. 参考解决方案
 - 基本框架
 - 词法分析器生成器Lex
 - SQL词法分析器
 - 语法分析器生成器Yacc
 - SQL语法分析器
 - 语法制导翻译设计
 - SQL语法制导翻译



1 上机题目

DBMS的设计与实现：

设计并实现一个DBMS原型系统，可以接受基本的SQL语句，对其进行词法分析、语法分析，然后解释执行SQL语句，完成对数据库文件的相应操作，实现DBMS的基本功能。

目的：

1. 加深编译原理基础知识的理解：词法分析、语法分析、语法制导翻译等；
2. 加深相关基础知识的理解：数据库系统、数据结构、操作系统等。



1 上机题目

基本SQL语句及功能：

- | | |
|--------------------|-------|
| 1. CREATE DATABASE | 创建数据库 |
| 2. USE DATABASE | 选择数据库 |
| 3. CREATE TABLE | 创建表 |
| 4. SHOW TABLES | 显示表名 |
| 5. INSERT | 插入元组 |
| 6. SELECT | 查询元组 |
| 7. UPDATE | 更新元组 |
| 8. DELETE | 删除元组 |
| 9. DROP TABLE | 删除表 |
| 10. DROP DATABASE | 删除数据库 |
| 11. EXIT | 退出系统 |

注：支持数据类型有INT、CHAR(N) 等

DBMS示例

测试程序



目录

1. 上机题目
2. 改进建议
3. 验收方式
4. 参考解决方案
 - 基本框架
 - 词法分析器生成器Lex
 - SQL词法分析器
 - 语法分析器生成器Yacc
 - SQL语法分析器
 - 语法制导翻译设计
 - SQL语法制导翻译



2 改进建议



改进建议

1. 安全；
2. 视图；
3. 索引（主码、外码、约束）；
4. 事务（日志）。

大致思路：

1. 设计词法规则；
2. 设计语法规则；
3. 设计数据库物理存储结构；
4. 设计语义规则；
5. 编程实现。



目录

1. 上机题目
2. 改进建议
3. 验收方式
4. 参考解决方案
 - 基本框架
 - 词法分析器生成器Lex
 - SQL词法分析器
 - 语法分析器生成器Yacc
 - SQL语法分析器
 - 语法制导翻译设计
 - SQL语法制导翻译



3 验收方式

条件:

1. 完成SQL词法分析器;
2. 完成SQL语法分析器;
3. 完成SQL语法制导翻译;

方法:

1. 演示程序的功能;
2. 解释程序的源代码;

成绩:

验收成绩 + 报告成绩(是否雷同) + 奖励分



目录

1. 上机题目
2. 改进建议
3. 验收方式
4. 参考解决方案
 - 基本框架
 - 词法分析器生成器Lex
 - SQL词法分析器
 - 语法分析器生成器Yacc
 - SQL语法分析器
 - 语法制导翻译设计
 - SQL语法制导翻译



4 参考解决方案

👉 基本框架

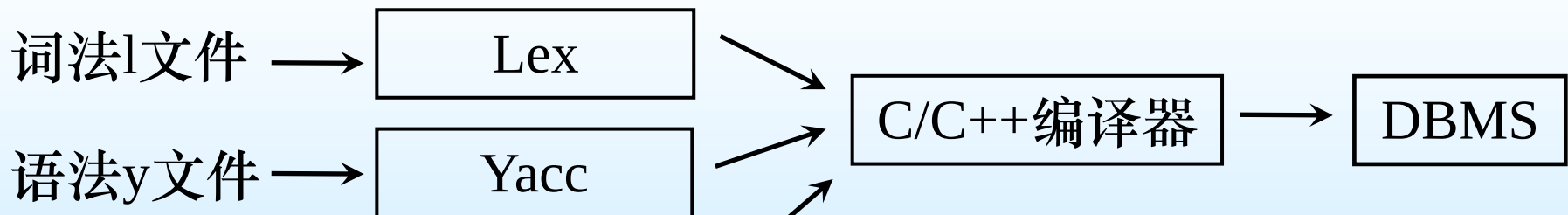


(a) DBMS工作原理

词法分析器
语法分析器
语义子程序



(b) 手工编写



语义子程序

(c) 工具生成

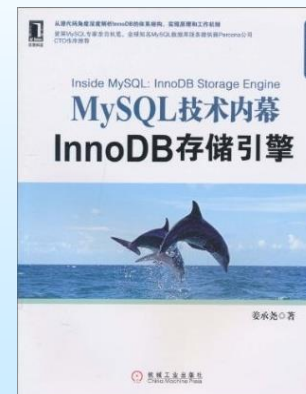
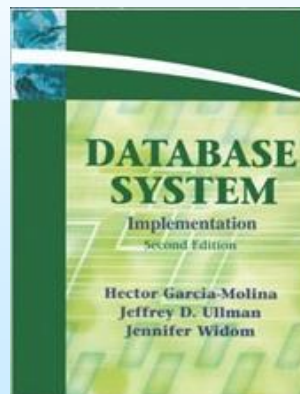
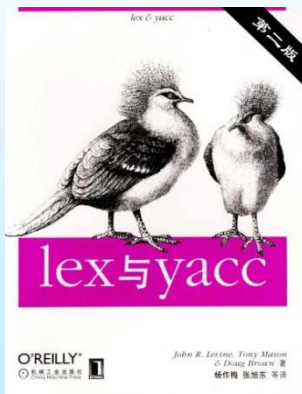
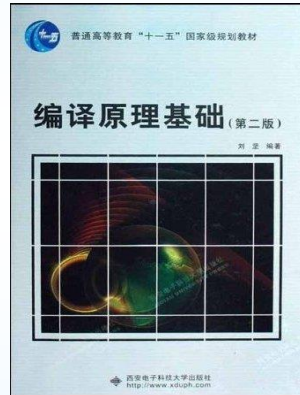
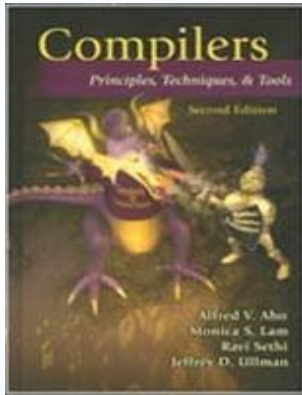


4 参考解决方案

工具:

1. 编译器: Visual C++
2. Lex与Yacc: Parser Generator (支持C/C++/Java, 带实例)

参考书:





目录

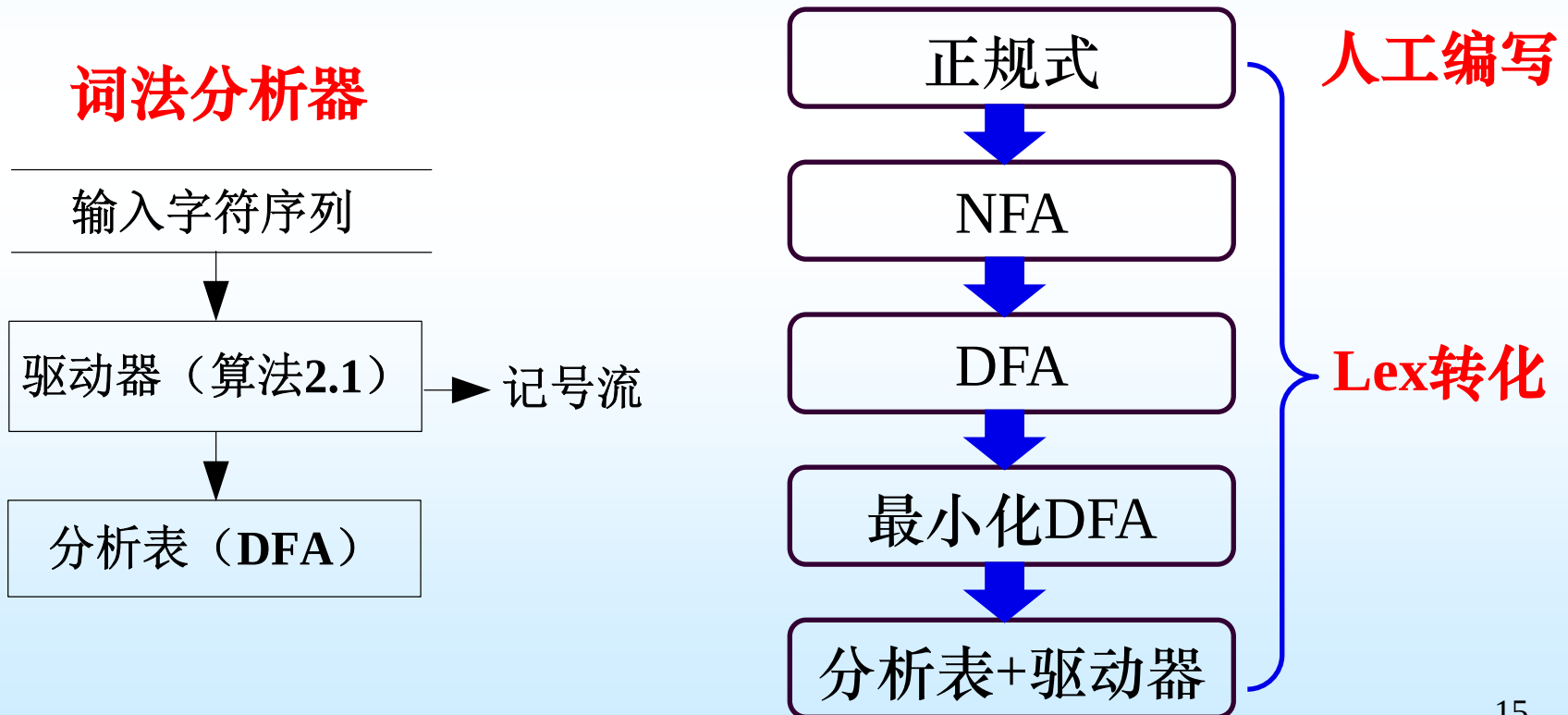
1. 上机题目
2. 改进建议
3. 验收方式
4. 参考解决方案
 - 基本框架
 - 词法分析器生成器Lex
 - SQL词法分析器
 - 语法分析器生成器Yacc
 - SQL语法分析器
 - 语法制导翻译设计
 - SQL语法制导翻译



4 参考解决方案

👉 词法分析器生成器Lex

Lex是一个词法分析器生成器，接受正规式表示的词法规则，生成识别正规式所描述语言的源程序，不同版本的Lex支持不同的高级语言，如C、C++、Java等。





4 参考解决方案

利用Lex构造词法分析器：实质为如何设计正规式和语义动作

Lex源程序*.l(三段式):

[定义

C声明

辅助定义正规式

]

%%

规则

词法规则正规式

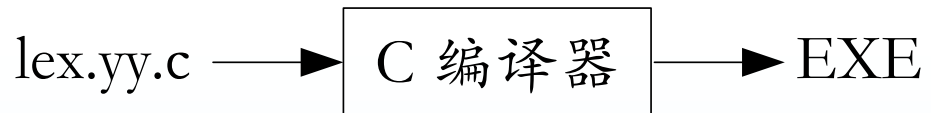
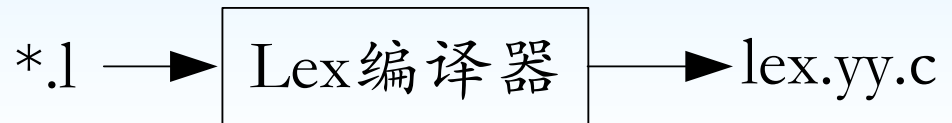
语义动作(C代码)

[%%

用户子程序

C源程序

]



注意：不同版本的Lex生成的文件不同



4 参考解决方案

Lex源程序*.l(三段式):

lex.yy.c结构:

[定义

C声明



(1)声明的C语言部分

辅助定义正规式



]

%%

规则

词法规则正规式



(2)词法分析表

语义动作(C代码)



(3)词法分析驱动器(yylex())

[%%

用户子程序

C源程序



(4)用户定义子程序

]



4 参考解决方案

实例1：标识符数字识别器—仅使用Lex

基本步骤：

1. 安装Parser Generator、Visual C++；
2. 分别配置Parser Generator、Visual C++；
3. 使用Parser Generator创建一个工程number
 - 编写l文件mylexer.l；
 - 编译mylexer.l，生成mylexer.h与mylexer.c；
4. 使用VC++创建Win32 Console Application工程number；
 - 配置该项目；
 - 加入mylexer.h与mylexer.c，编译工程；
 - 执行标识符数字识别器；

注意：每次修改l文件后，需要重新编译l文件，再重新编译VC工程



4 参考解决方案

mylexer.l源代码:

//声明部分

%{

#define ID 0

#define NUMBER 1

%}

//C声明：记号类别

将来在Yacc中使用下列语句代替：

%Token ID NUMBER

char [a-zA-Z]

digit [0-9]

digits {digit}+

optional_fraction ("."{digits})?

optional_exponent (E[+-]?{digits})?

//辅助定义正规式



4 参考解决方案

%%

{char}({char}|{digit})*

//词法规则正规式

{ printf("识别标识符%s: 长度为%d\n", yytext, yyleng);

return ID;

//语义动作: C代码

}

{digits}{optional_fraction}{optional_exponent}

{printf("识别数字%s: 长度为%d\n", yytext, yyleng);

return NUMBER;

}

yytext, yyleng: 全局变量, 用来存放识别出的输入序列, 由词法分析器自动填写

%%

int main(void)

{

//C源程序

printf("词法分析成功, 返回记号类别为%d\n", yylex());

}

yylex(): 词法分析驱动器, 根据词法规则返回记号的类别, 如ID、NUMBER等。²⁰



4 参考解决方案

执行结果：每次输入一个字符串，输出识别结果

```
abc12
识别标识符abc12：长度为5
词法分析成功，返回记号类别为0
Press any key to continue_
```

```
123
识别标识符123：长度为3
词法分析成功，返回记号类别为1
Press any key to continue_
```

注意：将来Yacc自动调用yylex()返回一个记号



目录

1. 上机题目
2. 改进建议
3. 验收方式
4. 参考解决方案
 - 基本框架
 - 词法分析器生成器Lex
 - SQL词法分析器
 - 语法分析器生成器Yacc
 - SQL语法分析器
 - 语法制导翻译设计
 - SQL语法制导翻译



4 参考解决方案

👉 SQL词法分析器

1. 词法分析器的三个任务：
 - 滤掉源程序中的无用成分；
 - 输出记号供语法分析器使用；
 - 识别非法输入，标记为出错记号
2. SQL记号的分类：关键字、标识符、数字、运算等。
3. 记号的组成：记号的类别和属性

记号	示例	类别	属性
关键字	CREATE、TABLE等	1	字符串
标识符	Student、Course等	2	字符串
数字	2、9、1.23等	3	数值
运算	+, -, *, /、sqrt等	4	函数

Yacc使用%union定义全局变量yylval的数据结构对应该表



4 参考解决方案

[例1] CREATE TABLE Student
 (Sno CHAR(9),
 Sname CHAR(20),
 Ssex CHAR(2),
 Sage INT
);

关键字： CREATE TABLE CHAR INT

标识符： Student Sno Sname Ssex Sage

数字： 9 20 2

其它符号： () , ;



4 参考解决方案

LEX源程序:

```
CREATE      {return CREATE;}
TABLE      {return TABLE;}
CHAR        {return CHAR;}
INT         {return INT;}
```

```
[A-Za-z][A-Za-z0-9_]*      {
    yylval.yych=(char *)malloc(strlen(yytext)+1);
    strcpy(yylval.yych, yytext);
    return ID;}
```

```
[0-9]+      {
    yylval.yych=(char *)malloc(strlen(yytext)+1);
    strcpy(yylval.yych, yytext);
    return NUMBER; }
```

引入Yacc后再起作用



4 参考解决方案

```
";" |  
"(" |  
")" |  
", " {return yytext[0]; }
```

CREATE TABLE Student

```
( Sno CHAR(9),  
  Sname CHAR(20),  
  Ssex CHAR(2),  
  Sage INT  
);
```



CREATE TABLE ID

```
(ID CHAR(NUMBER),  
 ID CHAR(NUMBER),  
 ID CHAR(NUMBER),  
 ID INT  
);
```

问题：如何实现大小写不敏感？



目录

1. 上机题目
2. 改进建议
3. 验收方式
4. 参考解决方案
 - 基本框架
 - 词法分析器生成器Lex
 - SQL词法分析器
 - 语法分析器生成器Yacc
 - SQL语法分析器
 - 语法制导翻译设计
 - SQL语法制导翻译

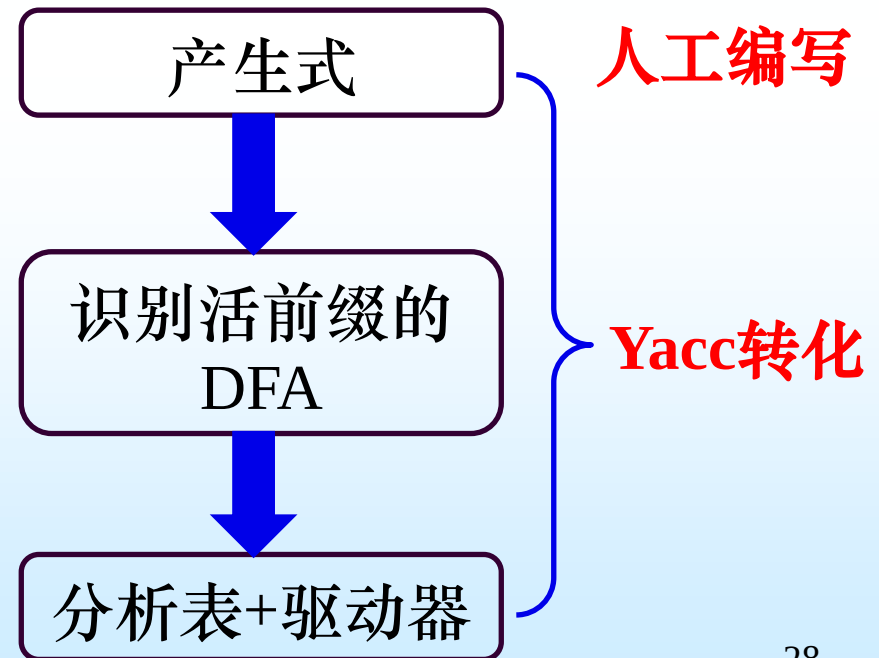
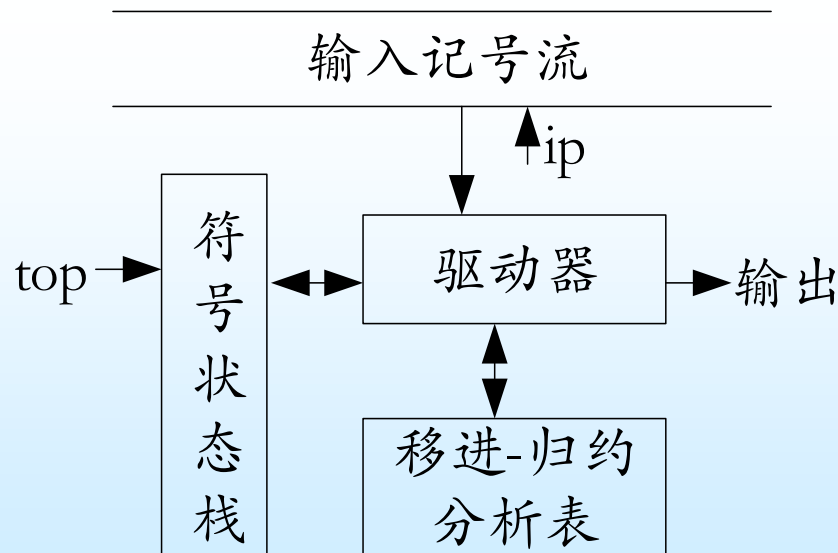


4 参考解决方案

👉 语法分析器生成器Yacc

Yacc是一个语法分析器生成器，接受产生式表示的语法规则，生成识别产生式所描述语言的源程序，不同版本的Yacc支持不同的高级语言，如C、C++、Java等。

语法分析器





4 参考解决方案

利用Yacc构造语法分析器：实质为如何设计产生式和语义动作

Yacc源程序*.y(三段式):

[定义

C声明

符号定义

]

%%

规则

语法规则产生式

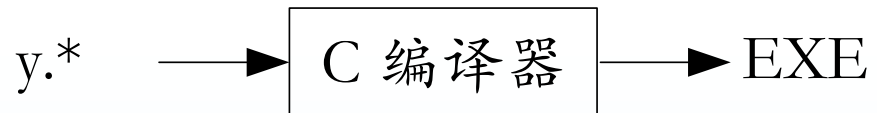
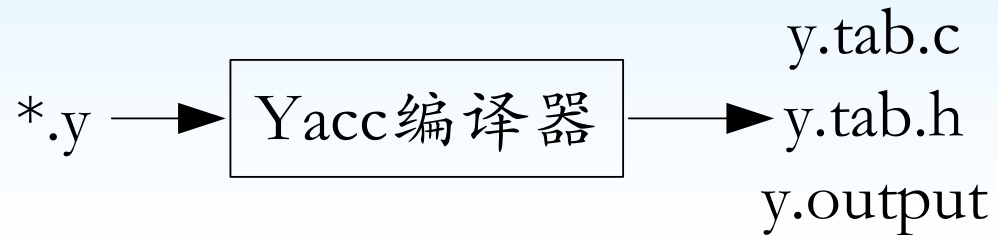
语义动作(C代码)

[%%

用户子程序

C源程序

]



注意：不同版本的Yacc生成的文件不同



4 参考解决方案

Yacc源程序*.y(三段式):

y.tab.c结构:

[定义

C声明

符号定义

]

%%

规则

语法规则产生式

语义动作(C代码)

[%%

用户子程序

C源程序

]

(1)声明的C语言部分

(2)语法分析表

(3)语法分析驱动器(yyparser())

(4)用户定义子程序



4 参考解决方案

实例2：算术运算表达式识别器—仅使用Yacc

基本步骤：

1. 使用Parser Generator创建一个工程calc
 - 编写y文件myparser.y;
 - 编译myparser.y，生成myparser.h与myparser.c;
2. 使用VC++创建Win32 Console Application工程calc
 - 配置该项目;
 - 加入myparser.h与myparser.c，编译工程;
 - 执行算术运算表达式识别器;

注意：每次修改y文件后，需要重新编译y文件，再重新编译VC工程



4 参考解决方案

myparser.y源代码:

```
%{
```

```
#include <ctype.h>
```

//C声明

```
%}
```

```
%token NUMBER
```

// 无优先级与结合性

//符号定义

```
%left '+', '-'
```

// 低优先级与左结合

%token,%left说明对应

```
%left '*', '/'
```

// 高优先级与左结合

实例1中的define

```
%%
```

```
expr      : expr '+' expr    { printf("识别加法!\n"); }
```

```
          | expr '-' expr    { printf("识别减法!\n"); }
```

```
          | expr '*' expr    { printf("识别乘法!\n"); }
```

```
          | expr '/' expr    { printf("识别除法!\n"); }
```

```
          | '(' expr ')'     { printf("识别括号!\n"); }
```

```
          | NUMBER          { printf("识别数字!\n"); }
```

```
;
```

//语法规则产生式
//语义动作



4 参考解决方案

```
%%
```

//C源程序

```
int main(void)
{ return yyparse();}
```

yyparse(): yacc自动生成的语法分析驱动器, 根据语法规则进行自下而上分析 (调用yylex进行词法分析), 同时执行语义动作。

```
int yylex(void)
```

yylex(): 用户自定义的词法分析驱动器。

```
{ int c;
  while ((c = getchar()) == ' ');//过滤空格
  if (isdigit(c)) {//判断是否是数字
    ungetc(c, stdin);
    scanf("%d", &yylval);
    return NUMBER;
  }
  if (c == '\n') return 0;//回车返回0, 代表词法分析器读取完毕
  return c;
}
```

yylval: 全局变量, 用来存放识别出的输入序列。



4 参考解决方案

执行结果：每次输入一个算术运算表达式，输出分析结果

```
1 + 2 + 3
识别数字!
识别数字!
识别加法!
识别数字!
识别加法!
Press any key to continue_
```

```
1 + 2 * 3
识别数字!
识别数字!
识别数字!
识别乘法!
识别加法!
Press any key to continue
```

注意： Yacc自动调用yylex()返回一个记号



4 参考解决方案

实例3：算术运算表达式识别器—同时使用Lex与Yacc

基本步骤：

1. 使用Parser Generator创建一个工程calc2
 - 编写l文件mylexer.l;
 - 编写y文件myparser.y;
 - 编译mylexer.l, 生成mylexer.h与mylexer.c;
 - 编译myparser.y, 生成myparser.h与myparser.c;
2. 使用VC++创建Win32 Console Application工程calc2
 - 配置该项目;
 - 加入mylexer.h与mylexer.c, myparser.h与myparser.c, 编译工程;
 - 执行算术运算表达式识别器;

注意：每次修改l、y文件后，需要重新编译l、y文件，再重新编译VC工程



4 参考解决方案

mylexer.l主要源代码:

digit [0-9]

digits {digit}+

optional_fraction ("."{digits})?

optional_exponent (E[+-]?{digits})?

%%

[]+ ; //过滤空格

{digits}{optional_fraction}{optional_exponent} { return NUMBER; }

"+" { return '+'; }

"-" { return '-'; }

"*" { return '*'; }

"/" { return '/'; }

"(" { return '('; }

")" { return ')'; }

"\n" { return 0; //回车返回0, 代表词法分析器读取完毕}



4 参考解决方案

mylexer.y主要源代码:

```
%token NUMBER // 无优先级与结合性
```

```
%left "+", "-" // 低优先级与左结合
```

```
%left "/*", "/" // 高优先级与左结合
```

```
%%
```

```
expr      : expr "+" expr      { printf("识别加法!\n"); }
```

```
          | expr "-" expr      { printf("识别减法!\n"); }
```

```
          | expr "*" expr      { printf("识别乘法!\n"); }
```

```
          | expr "/" expr      { printf("识别除法!\n"); }
```

```
          | "(" expr ")"       { printf("识别括号!\n"); }
```

```
          | NUMBER             { printf("识别数字!\n"); }
```

```
;
```

```
%%
```

```
int main(void)
```

```
{ return yyparse(); }
```

//符号定义

%token,%left说明对应
实例1中的define



4 参考解决方案

执行结果：每次输入一个算术运算表达式，输出分析结果

```
1.0 + 2.3 + 4.5
```

```
识别数字!
```

```
识别数字!
```

```
识别加法!
```

```
识别数字!
```

```
识别加法!
```

```
Press any key to continue
```

```
1.0 + 2.3 * 4.5
```

```
识别数字!
```

```
识别数字!
```

```
识别数字!
```

```
识别乘法!
```

```
识别加法!
```

```
Press any key to continue_
```

注意： Yacc自动调用Lex返回一个记号



目录

1. 上机题目
2. 改进建议
3. 验收方式
4. 参考解决方案
 - 基本框架
 - 词法分析器生成器Lex
 - SQL词法分析器
 - 语法分析器生成器Yacc
 - SQL语法分析器
 - 语法制导翻译设计
 - SQL语法制导翻译



4 参考解决方案

SQL语法分析器

1. 语法分析器的两个任务：
 - 为句子构造语法树；
 - 检查输入序列中的错误。
2. 主要工作：
 - 设计SQL的语法；
 - 定义产生式规则；
 - 设计测试用例，检验语法分析器是否正确。



4 参考解决方案

SQL语法: **注意: 多句执行或单句执行**

statements: statements statement | statement

statement: createsql | selectsql | insertsql | deletesql | updatesql ...

CREATE语句语法:

CREATE TABLE Student

(Sno CHAR(9), Sname CHAR(20), Ssex CHAR(2), Sage INT);

createsql: CREATE TABLE table '(' fieldsdefinition ')' ';' ;

table: ID

fieldsdefinition: field_type | fieldsdefinition ',' field_type

field_type: field type

field: ID

type: CHAR '(' NUMBER ')' | INT



4 参考解决方案

SELECT语句语法:

SELECT Sno, Sname FROM student WHERE Ssex='男' AND
Sage=20;

selectsql: SELECT fields_star FROM tables ';'

| SELECT fields_star FROM tables WHERE conditions ';'

fields_star: table_fields | '*'

table_fields: table_field | table_fields ',' table_field

table_field: field | table '.' field

tables: tables ',' table | table

conditions: condition | '(' conditions ')'

| conditions AND conditions | conditions OR conditions

condition: comp_left comp_op comp_right

comp_left: table_field

comp_right: table_field | NUMBER

comp_op: '<' | '>' | '=' | '!' '='

%left OR
%left AND



目录

1. 上机题目
2. 改进建议
3. 验收方式
4. 参考解决方案
 - 基本框架
 - 词法分析器生成器Lex
 - SQL词法分析器
 - 语法分析器生成器Yacc
 - SQL语法分析器
 - 语法制导翻译设计
 - SQL语法制导翻译



4 参考解决方案

☞ 语法制导翻译设计

1. 语法制导翻译的任务

对语义进行处理，以语法分析为基础，在语法分析得到语言结构的结果时，对附着于此结构的语义进行处理，如计算表达式的值、生成中间代码等。

2. 基本步骤

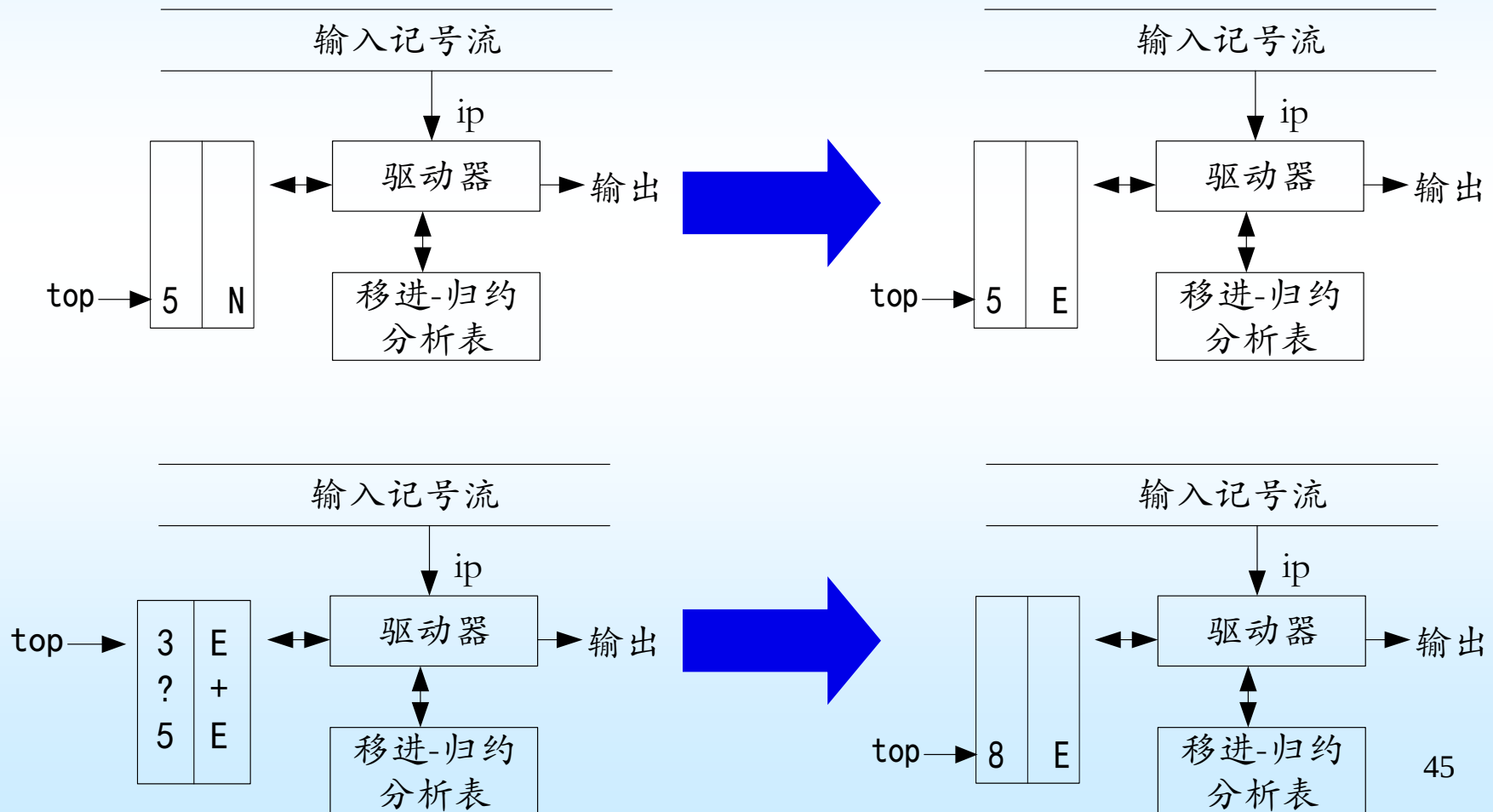
- 文法符号属性的设计
- 必要的基本操作（函数等）的设计
- 语义规则的设计



4 参考解决方案

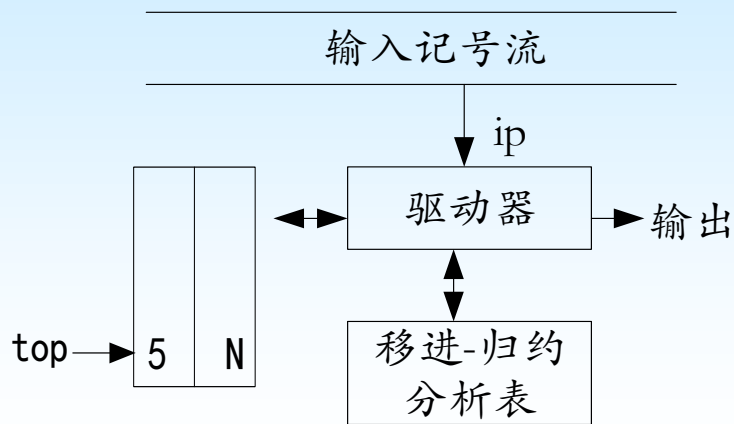
➤ 文法符号属性的设计

1. 属性说明





4 参考解决方案



N: 终结符NUMBER
5: 终结符属性

产生过程:

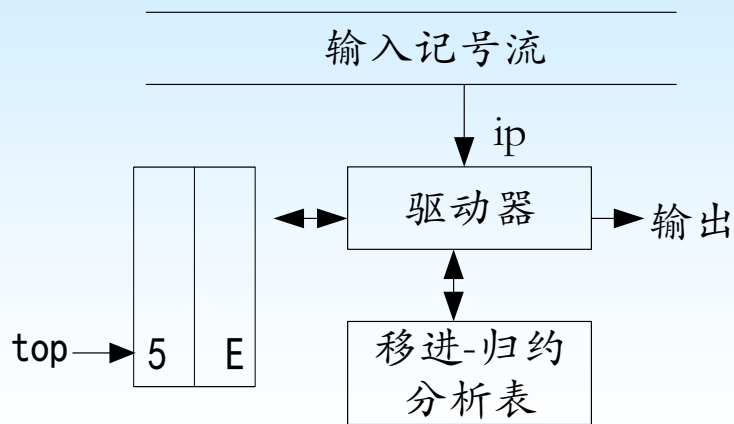
1. `yyparse()`调用`yylex()`;
2. `yylex()`分析字符串5, 返回记号(记号也称为终结符)类别NUMBER, 和其属性5;
3. `yyparse()`将终结符NUMBER移进符号栈, 将其属性5移进语义栈;

分析结果:

终结符NUMBER含有表示其值的属性(应该由Lex填写)



4 参考解决方案



E: 非终结符expr

5: 非终结符属性

产生过程:

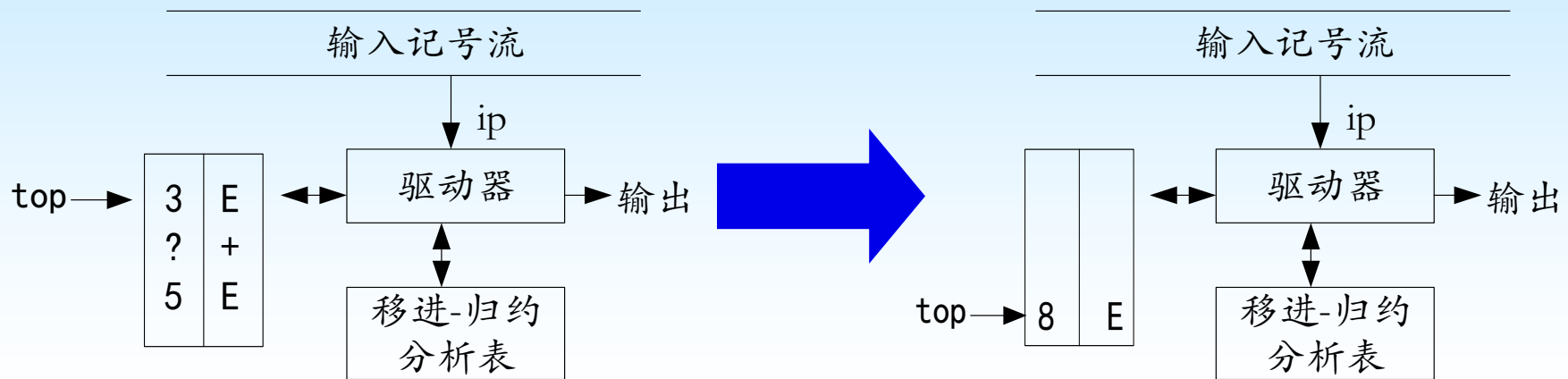
1. `yyparse()`判断符号栈顶端出现句柄N，需要规约为E；
2. 弹出符号栈中的句柄N及语义栈中对应的属性值5；
3. 将非终结符E压入符号栈，将对应的属性值5(综合属性，来自于N的属性)压入语义栈；

分析结果:

非终结符expr含有表示其值的属性(应该由Yacc填写)



4 参考解决方案



产生过程:

1. `yyparse()`判断符号栈顶端出现句柄 $E+E$ ，需要规约为 E ；
2. 弹出符号栈中的句柄 $E+E$ ，及语义栈中对应的属性值 $3?5$ ；
3. 将非终结符 E 压入符号栈，将对应的属性值 8 (综合属性，来自于 $E+E$ 的属性)压入语义栈；

分析结果:

属性值的计算应该由Yacc实现



4 参考解决方案

2. 终结符说明

%token: 终结符说明, 在Parser Generator中可使用%term
为了消除二义性, 可用如下说明代替%token:

- * %left: 左结合
- * %right: 右结合
- * %nonassoc: 无结合性

例如:

```
%term NUMBER
```

```
%left "+", "-"
```

```
%left "*", "/"
```

注: NUMBER为终结符, */优先级比+-高, 都是左结合
此时所有终结符的属性**默认都是int类型**



4 参考解决方案

更改终结符的属性类型

- 在Yacc中使用%union定义一个union结构

```
%union{    double nvalue;    }
```

相当于: typedef union{ double nvalue; } YYSTYPE;

覆盖了: #define YYSTYPE int;

因为YYSTYPE yylval;, 则yylval的类型发生改变

- 在Yacc中将该属性绑定到终结符NUMBER

```
%term <nvalue> NUMBER
```

访问NUMBER相当于访问**nvalue**



4 参考解决方案

3. 非终结符说明

%type: 非终结符说明, 在Parser Generator中可使用%nonterm

例如: %nonterm expr

注: expr为非终结符, 属性默认是int类型

更改非终结符的属性类型

☞ 在Yacc中使用%union修改原来的union结构

```
%union{    double nvalue;//保存NUMBER的值
        struct{ double evalue;//保存expr的值
                char post[1024]);//保存expr的后缀式
        }property;    }
```

☞ 在Yacc中将该属性绑定到非终结符expr

```
%nonterm <property> expr
```

访问expr相当于访问property



4 参考解决方案

➤ 必要的基本操作（函数等）的设计

根据需要自定义函数

```
double MyAdd(double x, double y)
{
    return x+y;
}
```



4 参考解决方案

➤ 语法规则的设计

☞ 在Lex中对终结符属性进行赋值

```
{digits}{optional_fraction}{optional_exponent}  
    {  
        yylval.nvalue = strtod(yytext, &stop);  
        return NUMBER;  
    }
```

注：yytext、yylength为全局变量，由Lex自动填写，分别保存记号的字面量与长度，由用户读取；yylval为全局变量，类型为%union定义的数据结构，由用户填写，由Yacc读取

```
%union{  
    double nvalue; //保存NUMBER的值  
    struct{ double evalue; //保存expr的值  
            char post[1024]; //保存expr的后缀式  
        }property;  
}
```



4 参考解决方案

在Yacc中对非终结符属性进行赋值

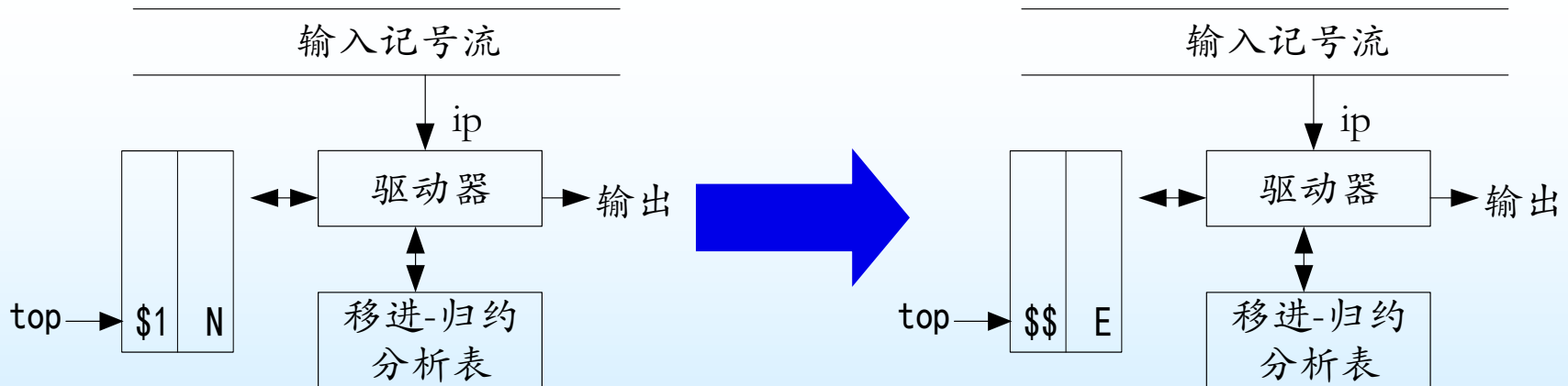
```
expr: NUMBER {  
    $1类型  
    double nvalue
```

\$\$类型

struct property

```
    $$.$value = $1;  
    sprintf($$.post,"%E",$$.$value);  
}
```

```
double nvalue;  
struct{double evalue;  
        char post[1024];  
    }property;
```





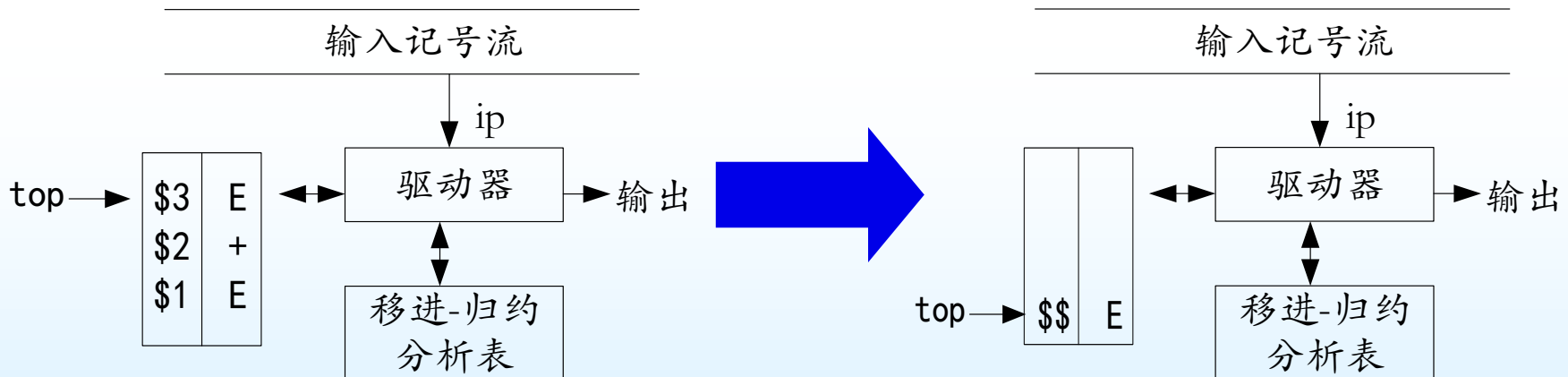
4 参考解决方案

```
expr: expr "+" expr
      {
```

**\$\$, \$1, \$3类型
struct property**

```
    $$.$value = MyAdd($1.$value, $3.$value);
    sprintf($$.post, "%s %s +", $1.post, $3.post);
  }
```

```
double nvalue;
struct{double evalule;
        char post[1024];
        }property;
```





4 参考解决方案

实例4：算术运算表达式的求值与后缀式(实例3的改进)

基本步骤：

1. 使用Parser Generator创建一个工程calc3
 - 编写l文件mylexer.l;
 - 编写y文件myparser.y;
 - 编译mylexer.l, 生成mylexer.h与mylexer.c;
 - 编译myparser.y, 生成myparser.h与myparser.c;
2. 使用VC++创建Win32 Console Application工程calc3
 - 配置该项目;
 - 加入mylexer.h与mylexer.c, myparser.h与myparser.c, 编译工程;
 - 执行算术运算表达式识别器;

注意：每次修改l、y文件后，需要重新编译l、y文件，再重新编译VC工程



4 参考解决方案

mylexer.l源代码

```
%{  
    #include "myparser.h"  
    #include <stdlib.h>  
    char * stop;  
%}  
  
//辅助定义正规式  
digit    [0-9]  
digits   {digit}+  
optional_fraction    ("."{digits})?  
optional_exponent    (E[+-]?{digits})?  
%%
```



4 参考解决方案

//词法规则正规式

[]+ ;

{digits}{optional_fraction}{optional_exponent}

```
{  
    yylval.nvalue = strtod(yytext, &stop); //将字面量转化为值  
    return NUMBER;  
}
```

```
"+" { return '+'; }
```

```
"-" { return '-'; }
```

```
"*" { return '*'; }
```

```
"/" { return '/'; }
```

```
"(" { return '('; }
```

```
")" { return ')'; }
```

```
"\n" { return 0; }
```

```
%%
```



4 参考解决方案

myparser.y源代码

```
%{  
    #include "mylexer.h"  
    #include <ctype.h>  
    #include <stdlib.h>  
    double MyAdd(double x, double y); //声明自定义函数  
%}  
%union{ //重定义YYSTYPE  
    double nvalue;           //终结符NUMBER的属性  
    struct{                   //非终结符expr的组合属性  
        double eval;         // expr值  
        char post[1024];     // expr后缀式  
    } property;  
}
```



4 参考解决方案

```
%nonterm <property> expr //绑定非终结符expr的属性
%term <nvalue> NUMBER //绑定终结符NUMBER的属性
%left "+", "-" //低优先级与左结合
%left "*/", "/" //高优先级与左结合
%%
start : expr
      { //新加产生式，规约时打印表达式值与后缀式
        printf("值: %E\n", $1.evalue);
        printf("后缀式: %s\n", $1.post);
      };
expr : expr "+" expr
     { $$evalue = MyAdd($1.evalue, $3.evalue);
       sprintf($$.post, "%s %s +", $1.post, $3.post);
     }
```



4 参考解决方案

| expr "-" expr

```
{    $$eval = $1eval - $3eval;
    printf($$.post,"%s %s -",$1.post,$3.post);
}
```

| expr "*" expr

```
{    $$eval = $1eval * $3eval;
    printf($$.post,"%s %s *",$1.post,$3.post);
}
```

| expr "/" expr

```
{    $$eval = $1eval / $3eval;
    printf($$.post,"%s %s /",$1.post,$3.post);
}
```



4 参考解决方案

```
| "(" expr ")"  
{  
    $$eval = $2.eval;  
    printf($$.post,"%s",$2.post);  
}  
| NUMBER  
{  
    $$eval = $1;  
    printf($$.post,"%E",$$eval);  
};
```

%%

double MyAdd(double x, double y)//自定义函数

```
{ return x+y; }
```

int main(void)

```
{ return yyparse(); }
```



4 参考解决方案

执行结果：每次输入一个算术运算表达式，输出执行结果

```
1e2+2.4e4*6.7e5  
值: 1.608000E+010  
后缀式: 1.000000E+002 2.400000E+004 6.700000E+005 * +  
Press any key to continue_
```

$1e2+2.4e4*6.7e5$

值: $1.608000E+010$

后缀式: $1.000000E+002\ 2.400000E+004\ 6.700000E+005\ * +$



目录

1. 上机题目
2. 改进建议
3. 验收方式
4. 参考解决方案
 - 基本框架
 - 词法分析器生成器Lex
 - SQL词法分析器
 - 语法分析器生成器Yacc
 - SQL语法分析器
 - 语法制导翻译设计
 - SQL语法制导翻译



4 参考解决方案

👉 SQL语法制导翻译

1. SQL语法制导翻译的任务

以SQL语法分析为基础，在语法分析得到语法树结果时，对附着于此结构的语义进行处理，**将SQL语句转化为对数据库物理文件的操作。**

2. 基本步骤

- 文法符号属性的设计
- **数据库物理结构的设计**
- 必要的基本操作（函数等）的设计
- 语义规则的设计
- **设计测试用例**

注：设计数据库的物理结构是实现DBMS的前提条件，基础知识来源于操作系统、数据结构等。



4 参考解决方案

➤ 文法符号属性的设计

CREATE语句语法树节点设计:

CREATE TABLE Student

(Sno CHAR(9), Sname CHAR(20), Ssex CHAR(2), Sage INT);

createsql: CREATE TABLE **table** '(' **fieldsdefinition** ')' ';' ;

table: ID

fieldsdefinition: field_type | fieldsdefinition ',' field_type

field_type: field type

field: ID

type: CHAR '(' NUMBER ')' | INT

分析结果: CREATE语法树需要保存非终结符table、fieldsdefinition的相关属性, 将来用于创建基本表



4 参考解决方案

数据结构设计:

```
enum Type {INT,CHAR};           //字段可用类型
struct Createfieldsdef{         /*create语句中的字段定义*/
    char                *field;  //字段名称
    enum TYPE           type;    //字段类型
    int                 length;  //字段长度
    struct Createfieldsdef *next_fdef; //下一字段
};

struct Createstruct{            /*create语法树根节点*/
    char *table;                //基本表名称
    struct Createfieldsdef      *fdef; //字段定义
};
```



4 参考解决方案

重定义YYSTYPE:

```
%union                                     /*定义yylval的格式*/
{ /*属于create语法树的类型*/
    char * yych;                          //字面量
    struct Createfieldsdef *cfdef_var;      //字段定义
    struct Createstruct *cs_var;          //整个create语句
}
```

绑定到终结符、非终结符:

```
/*create语句中涉及的符号*/
%term      CREATE TABLE ID CHAR NUMBER INT
%nonterm   <yych>          table   field   type
%nonterm   <cfdef_var>     fieldsdefinition field_type
%nonterm   <cs_var>        createsql
```



4 参考解决方案

最终需要生成的语法树：

CREATE TABLE Student

(Sno CHAR(9), Sname CHAR(20), Ssex CHAR(2), Sage INT);

table	fdef
Student	

field	type	length	next_fdef
Sno	CHAR	9	

field	type	length	next_fdef
Sname	CHAR	20	

field	type	length	next_fdef
Ssex	CHAR	2	

field	type	length	next_fdef
Sage	INT	4	NULL



4 参考解决方案

SELECT语句语法树节点设计:

**SELECT Sno, Sname FROM student WHERE Ssex='男' AND
Sage=20;**

selectsql: SELECT fields_star FROM tables ';'

| SELECT fields_star FROM tables WHERE conditions ';'

fields_star: table_fields | '*'

table_fields: table_field | table_fields ',' table_field

table_field: field | table '.' field

tables: tables ',' table | table

conditions: condition | '(' conditions ')'

| conditions AND conditions | conditions OR conditions

分析结果: SELECT语法树需要保存非终结符fields_star、
tables、conditions的相关属性, 将来用于查询基本表



4 参考解决方案

数据结构设计:

Ssex='男' AND Sage=20

```
struct Conditions{/*条件*/
    struct Conditions *left;           //左部条件
    struct Conditions *right;          //右部条件
    char comp_op;    /* 'a'是and, 'o'是or, '<' , '>' , '=' , '!=' */
    int  type; /* 0是字段, 1是字符串, 2是整数 */
    char *value; /* 根据type存放字段名、字符串或整数 */
    char *table; /* NULL或表名 */
};

struct Selectedfields{/*select语句中选中的字段*/
    char *table;           //字段所属表
    char *field;           //字段名称
    struct Selectedfields *next_sf; //下一个字段
};
```



4 参考解决方案

```
struct Selectedtables{           /*select语句中选中的表*/
    char *table;                 //基本表名称
    struct Selectedtables *next_st; //下一个表
};

struct Selectstruct{            /*select语法树的根节点*/
    struct Selectedfields *sf;   //所选字段
    struct Selectedtables *st;   //所选基本表
    struct Conditions *cons;    //条件
};
```




4 参考解决方案

重定义YYSTYPE:

```
%union                                           /*定义yylval的格式*/
{ /*属于select语法树的类型*/
    struct Selectedfields      *sf_var;          //所选字段
    struct Selectedtables      *st_var;          //所选表格
    struct Conditions          *cons_var;        //条件语句
    struct Selectstruct        *ss_var;          //整个select语句
}
```

绑定到终结符、非终结符:

/*select语句中涉及的部分非终结符*/

```
%nonterm <sf_var> fields_star table_fields table_field
```

```
%nonterm <st_var> tables
```

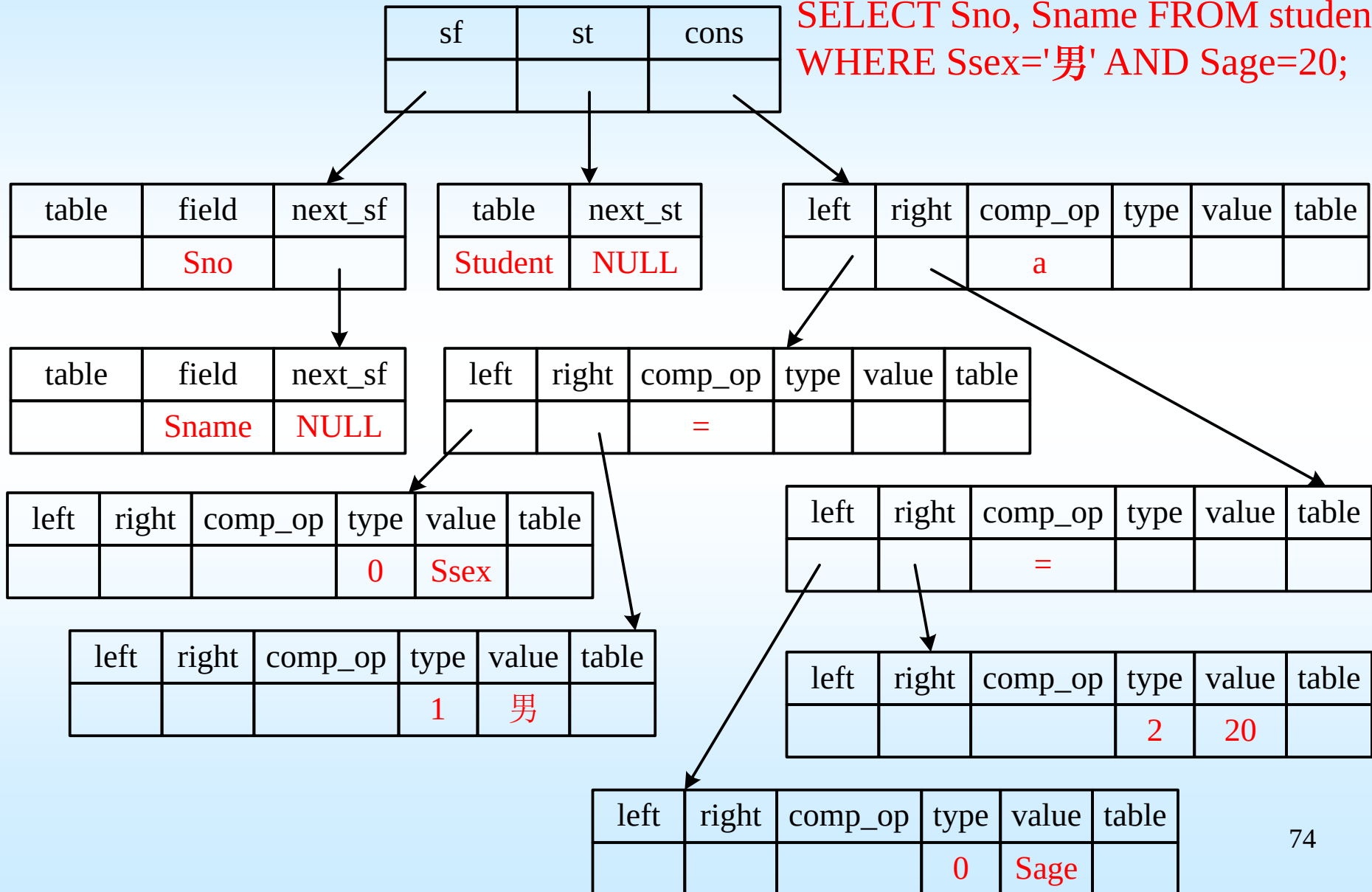
```
%nonterm <cons_var> condition conditions
```

```
%nonterm <ss_var> selectsql
```



4 参考解决方案

SELECT Sno, Sname FROM student
WHERE Ssex='男' AND Sage=20;





4 参考解决方案

➤ 数据库物理结构的设计

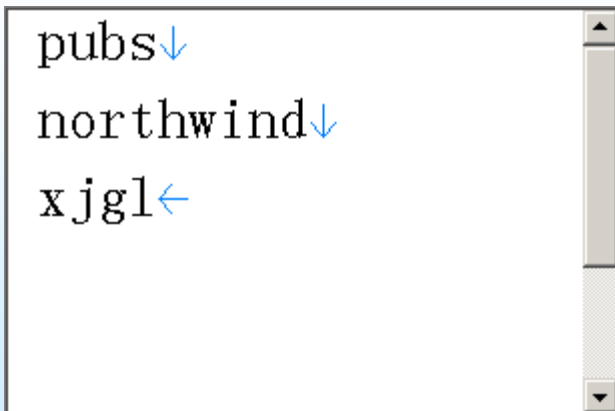
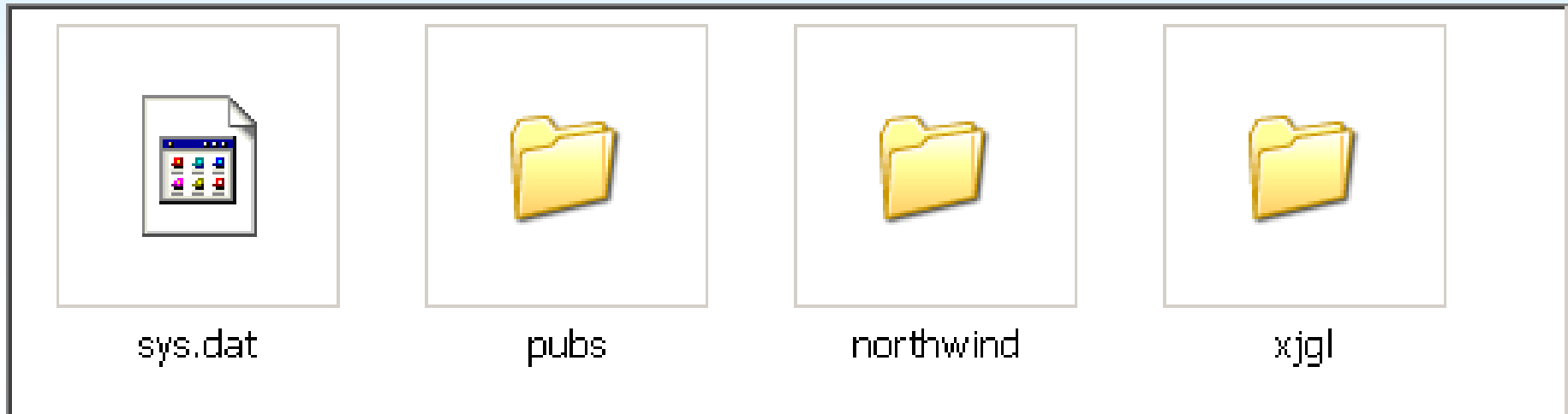
数据库信息分为元数据和基本数据：元数据是数据的类型信息，基本数据才是具体的数据。为了便于实现，作如下规定：

- ① 数据格式为文本，便于调试；
- ② 元数据文件为sys.dat，整个DBMS的sys.dat存放用户数据库的名字，每个用户数据库的sys.dat存放用户表每个字段的类型信息；
- ③ 基本数据文件后缀为txt，每个表的数据分开存放，如student表的数据存放在student.txt中；
- ④

注：物理结构根据需要自行设计

4 参考解决方案

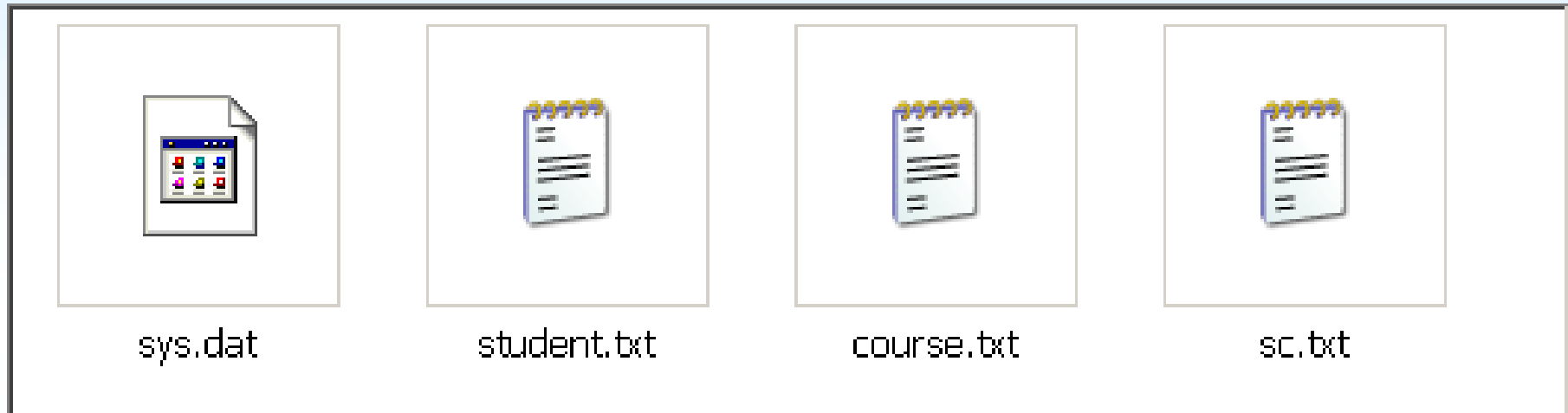
例如，DBMS主目录信息如下：



如左图，sys.dat中包含3行信息，表明包含3个用户数据库，同时对应3个目录

4 参考解决方案

例如，学籍管理数据库对应的xjgl目录信息如下：



xjgl数据库中包含3个表：student，course，sc，数据分别存放在student.txt，course.txt，sc.txt中



4 参考解决方案

sys.dat信息如下：

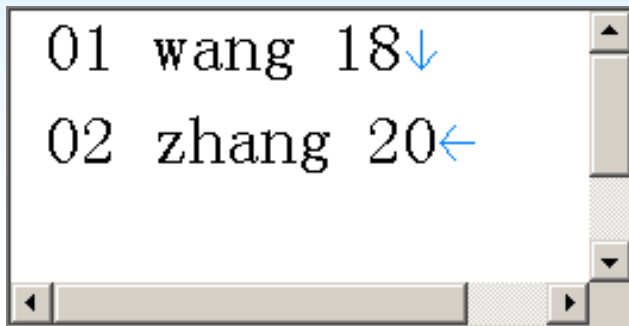
```
student 1 sno char 2↓  
student 2 sname char 20↓  
student 3 sage int↓  
course 1 cno char 2↓  
course 2 cname char 20↓  
course 3 credit int↓  
sc 1 sno char 2↓  
sc 2 cno char 2↓  
sc 3 grade int←
```

表明student，course，sc3个表均有3个字段，规定了各自的类型及长度。

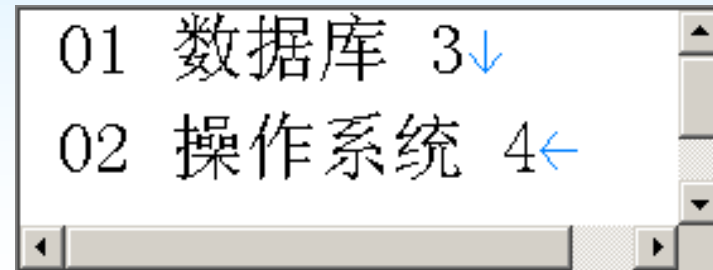


4 参考解决方案

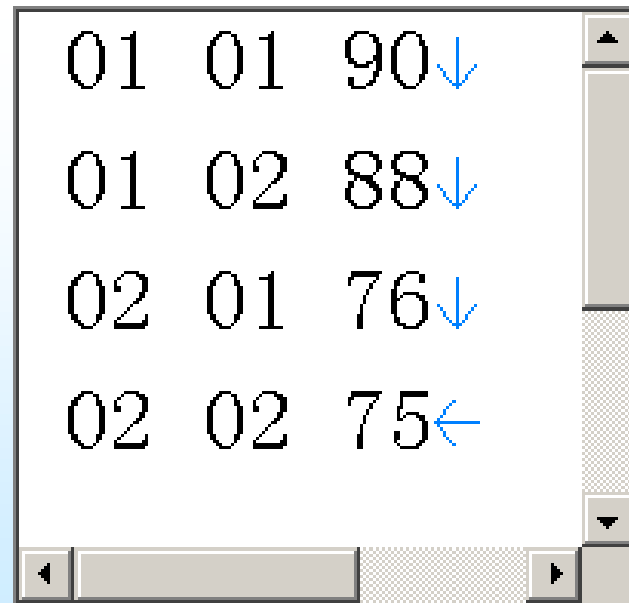
student.txt, course.txt, sc.txt的信息如下:



01 wang 18↓
02 zhang 20←



01 数据库 3↓
02 操作系统 4←



01 01 90↓
01 02 88↓
02 01 76↓
02 02 75←



4 参考解决方案

➤ 必要的基本操作（函数等）的设计

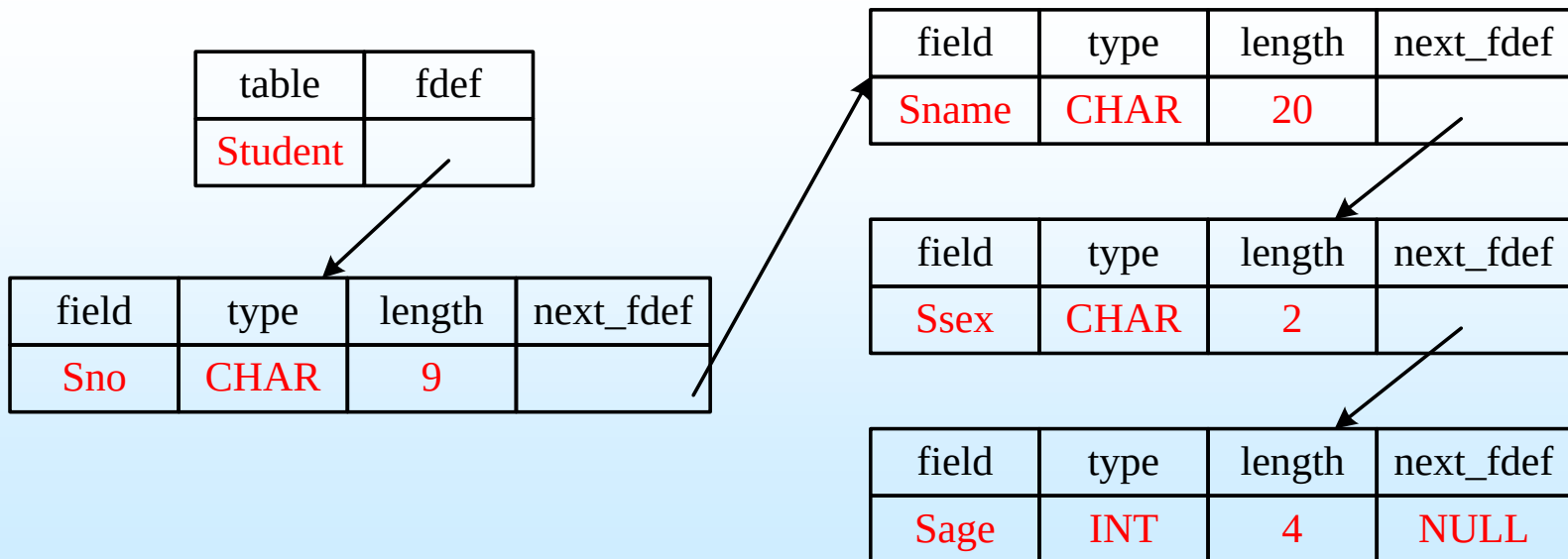
createtable函数

输入：create语句语法树根节点

输出：修改xjgl目录下的sys.dat，新建txt文件

1. 在sys.dat中加入student每个列的说明
2. 新建student.txt文件

```
CREATE TABLE Student(  
    Sno    CHAR(9),  
    Sname  CHAR(20),  
    Ssex   CHAR(2),  
    Sage   INT    );
```





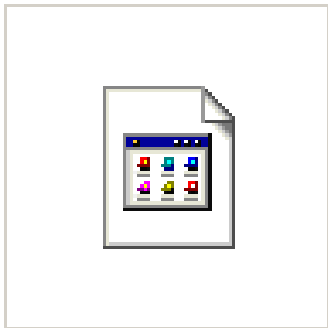
4 参考解决方案

sys.dat的内容如下:

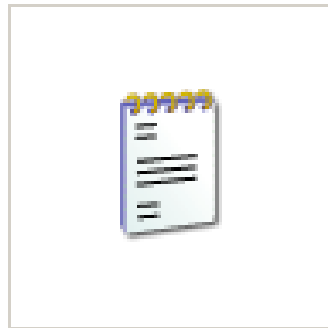
```
student 1 sno char 2↓  
student 2 sname char 20↓  
student 3 ssex char 2↓  
student 4 sage int↓
```

```
CREATE TABLE Student(  
    Sno    CHAR(9),  
    Sname  CHAR(20),  
    Ssex   CHAR(2),  
    Sage   INT    );
```

xjgl目录的内容如下:



sys.dat



student.txt



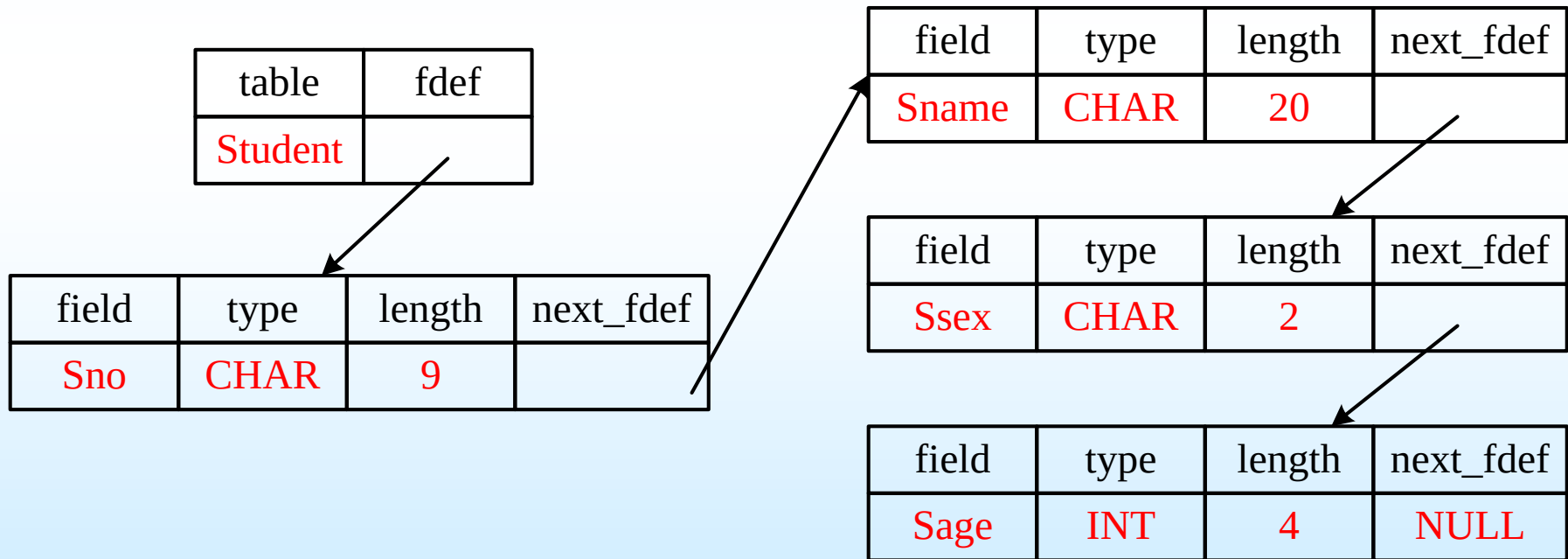
4 参考解决方案

➤ 语义规则的设计

CREATE TABLE Student(

Sno CHAR(9), Sname CHAR(20), Ssex CHAR(2), Sage INT);

根据语法树执行createtable函数





4 参考解决方案

CREATE TABLE Student(

Sno CHAR(9), Sname CHAR(20), Ssex CHAR(2), Sage INT);

field_type: field type

fieldsdefinition: field_type | fieldsdefinition ',' field_type

field	type	length	next_fdef
Sname	CHAR	20	

field	type	length	next_fdef
Ssex	CHAR	2	

field	type	length	next_fdef
Sage	INT	4	NULL

field	type	length	next_fdef
Sno	CHAR	9	



4 参考解决方案

createsql: CREATE TABLE table '(' fieldsdefinition ')' ';' ;

{

\$\$=(struct Createstruct *)malloc(sizeof(struct Createstruct));

\$\$->table=\$3;

\$\$->fdef=\$5;

}

table	fdef
Student	

field	type	length	next_fdef
Sname	CHAR	20	

field	type	length	next_fdef
Ssex	CHAR	2	

field	type	length	next_fdef
Sage	INT	4	NULL

field	type	length	next_fdef
Sno	CHAR	9	



4 参考解决方案

statement : createsql

```
{  int result;
  result=createtable($1);
  if(result==1){
    printf("Create Table %s Successful!\n",$1->table);
  }else
  {  printf("Create Table %s Failed!\n",$1->table);
  }
}
```

table	fdef
Student	

field	type	length	next_fdef
Sno	CHAR	9	

field	type	length	next_fdef
Sname	CHAR	20	

field	type	length	next_fdef
Ssex	CHAR	2	

field	type	length	next_fdef
Sage	INT	4	NULL



4 参考解决方案

➤ 设计测试用例

☞ 单条语句一一执行：简单

1. CREATE DATABASE
2. USE DATABASE
3. CREATE TABLE
4. INSERT
5. SELECT
6. UPDATE
7. DELETE
8. DROP TABLE
9. DROP DATABASE

创建数据库

选择数据库

创建表

插入元组

查询元组

更新元组

删除元组

删除表

删除数据库

☞ 多条语句连续执行：复杂



结束!
问题?